SSN COLLEGE OF ENGINEERING
KALAVAKKAM-603110

ICS1612-Big Data Management Laboratory

Mini Project

Project Title

Real-Time Defect Detection in Manufacturing Using Big Data Technologies

Team Members

Harini J, 3122237001015

Jayasree R , 3122237001019

1. Introduction

1.1 Overview of Big Data Analytics and Real-Time Data Processing

Big Data Analytics refers to the process of analyzing large volumes of structured and unstructured data to extract meaningful insights. With the rapid growth of data generated from sensors, machines, and digital systems, traditional processing methods are no longer sufficient. Real-time data processing enables systems to process and analyze data as it is generated, allowing immediate decision-making. This is especially important in applications where delays can lead to critical failures, such as industrial monitoring, fraud detection, and healthcare systems.

1.2 Importance of Scalable Big Data Systems

Modern data systems must be scalable to handle continuously increasing data volumes. Scalable systems can efficiently process large datasets without performance degradation. In industrial environments, machines generate data at high speeds, requiring systems that can handle high throughput, process data continuously, and maintain reliability and fault tolerance. Scalability ensures that the system can grow with increasing data demands without needing major redesign.

1.3 Role of Technologies such as Apache Spark and NoSQL Databases

Technologies like Apache Spark play a key role in enabling fast and distributed data processing. Spark provides in-memory computation and supports real-time data processing through streaming. NoSQL databases such as MongoDB are designed to handle large volumes of unstructured data and offer flexible schema design, high scalability, and fast read/write operations. These technologies together enable efficient handling of big data in real-time applications.

1.4 Brief Introduction to the Selected Application Domain

This project focuses on the manufacturing domain, specifically automated quality control. In high-speed production lines, manual inspection of products is inefficient and error-prone. To address this, automated systems are used to analyze images of products in real time and detect defects. This ensures improved product quality, reduced human effort, and faster inspection processes. The proposed system uses real-time data streaming and processing to detect defects in manufactured items such as screws.

2. Problem Statement

2.1 The Real-World Problem Being Addressed

To ensure product quality in manufacturing industries, automated inspection is essential. Traditional quality inspection methods rely on manual checking, which is time-consuming, inconsistent, and prone to human error. Defective products may go unnoticed, leading to customer dissatisfaction and financial loss. Therefore, there is a need for an automated system that can detect defects accurately and efficiently.

2.2 Why Large-Scale or Streaming Data Processing is Required

Modern production lines generate a continuous stream of data, especially image data from cameras monitoring products. Processing this data requires real-time analysis, high-speed data handling, and continuous processing without delays. Streaming systems allow data to be processed as it arrives, enabling immediate detection of defects and faster decision-making.

2.3 Limitations of Traditional Data Processing Systems

Traditional batch processing systems are not suitable for real-time applications because they process data in large batches after collection, introduce delays in analysis, and cannot handle continuous high-speed data streams efficiently. Such systems fail to provide instant insights required in manufacturing environments.

3. Objectives

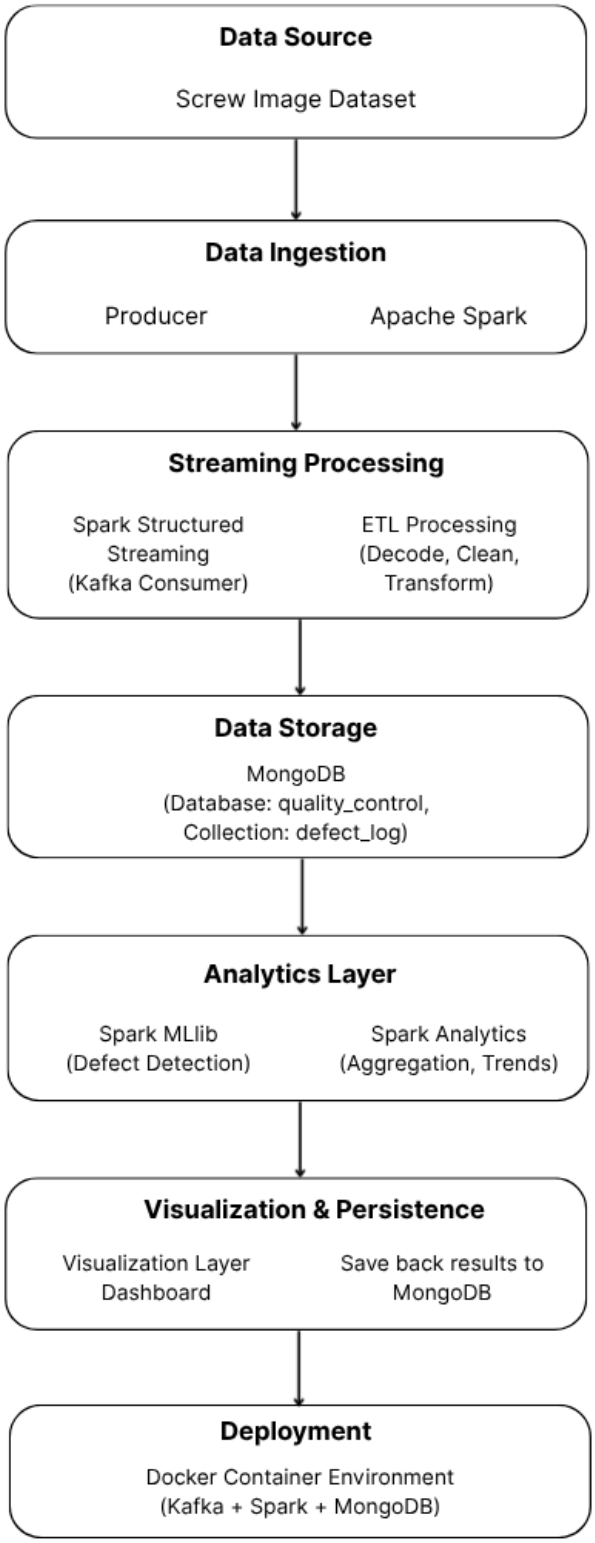
- Develop a Streaming Data Ingestion Pipeline using Apache Kafka
- Perform ETL Operations on Streaming Data using PySpark
- Implement Real-Time Defect Detection using Spark MLlib
- Store Processed Data in a Scalable NoSQL Database (MongoDB)
- Perform Real-Time Analytics on Defect Data
- Enable Visualization of Analytical Insights
- Build a Scalable and Efficient Big Data Processing Pipeline

4. Development Environment

Software Tools Used

- Apache Spark
  Used for real-time data processing and streaming analytics. It enables efficient handling of large-scale data and supports distributed computation.
- Spark MLlib
  Used for implementing machine learning models for defect detection and classification on streaming data.
- Python / PySpark
  Used for implementing the streaming pipeline, data processing logic, and integration between Kafka, Spark, and MongoDB.
- Apache Kafka
  Used for real-time data streaming and message queuing. It acts as a bridge between the data producer and consumer.
- MongoDB
  Used as a NoSQL database to store processed defect data. It supports flexible schema design and high scalability.
- OpenCV
  Used for basic image processing tasks such as decoding and handling image data.

5. System Architecture Diagram


Data Source
Screw Image Dataset
Data Ingestion
Producer
Apache Spark
Streaming Processing
Spark Structured Streaming (Kafka Consumer)
ETL Processing (Decode, Clean, Transform)
Data Storage
MongoDB (Database: quality_control, Collection: defect_log)
Analytics Layer
Spark MLlib (Defect Detection)
Spark Analytics (Aggregation, Trends)
Visualization & Persistence
Visualization Layer Dashboard
Save back results to MongoDB
Deployment
Docker Container Environment (Kafka + Spark + MongoDB)

6. Dataset Collection

6.1 Source of Data

The dataset used in this project is a screw defect dataset collected from an open dataset repository. It consists of images categorized into different classes such as good, scratch_head, scratch_neck, thread_side, thread_top, and manipulated_front. The dataset simulates real-time image input from a manufacturing production line.

Dataset source:
https://www.kaggle.com/datasets/ipythonx/mvtec-ad

The streaming data is simulated by continuously sending images from this dataset through Apache Kafka to mimic a real-time camera feed.

6.2 Data Format

The dataset consists of image files in JPG/PNG format. During streaming, the images are converted into Base64 encoded strings before being sent to Kafka. The processed data stored in MongoDB is in JSON format.

6.3 Volume and Velocity of Data

The dataset contains multiple categories of screw images with hundreds of samples in each class. The system simulates continuous streaming by sending images at regular intervals, representing high-velocity data similar to real-time industrial environments.

6.4 Example Sample Record

Example of processed record stored in MongoDB:

```
{
"product_id": "12_3",
"status": "defect",
"confidence": 0.87
}
```

7. Implementation with Screenshots

7.1 Data Collection / Streaming Data Ingestion

The streaming pipeline begins with data ingestion using Apache Kafka. A Python-based producer reads screw images from the dataset, encodes them into Base64 format, and sends them to a Kafka topic named "raw-sensor-images". This simulates a real-time camera capturing products on a production line.

Spark Structured Streaming is configured to consume data from the Kafka topic. It continuously listens to incoming messages and processes them in real time.

Key configuration parameters include Kafka bootstrap server address and topic subscription.

Example code snippet:

```
producer = KafkaProducer(bootstrap_servers='localhost:9092')

df = spark.readStream.format("kafka") \
.option("kafka.bootstrap.servers", "localhost:9092") \
.option("subscribe", "raw-sensor-images") \
.load()
```

Output Screenshots :

kafka:

```
[2026-03-30 08:03:56,939] INFO [Controller id=0, targetBrokerId=0] Node 0 disconnected. (org.apache.kafka.clients.Networ
kClient)
[2026-03-30 08:03:56,944] WARN [Controller id=0, targetBrokerId=0] Connection to node 0 (Harini.localdomain/127.0.1.1:90
92) could not be established. Node may not be available. (org.apache.kafka.clients.NetworkClient)
[2026-03-30 08:03:56,948] INFO [SocketServer listenerType=ZK_BROKER, nodeId=0] Enabling request processing. (kafka.netwo
rk.SocketServer)
[2026-03-30 08:03:56,949] INFO [Controller id=0, targetBrokerId=0] Client requested connection close from node 0 (org.ap
ache.kafka.clients.NetworkClient)
[2026-03-30 08:03:56,955] INFO Awaiting socket connections on 0.0.0.0:9092. (kafka.network.DataPlaneAcceptor)
[2026-03-30 08:03:56,971] INFO Kafka version: 3.7.0 (org.apache.kafka.common.utils.AppInfoParser)
[2026-03-30 08:03:56,971] INFO Kafka commitId: 2ae524ed625438c5 (org.apache.kafka.common.utils.AppInfoParser)
[2026-03-30 08:03:56,972] INFO Kafka startTimeMs: 1774838036960 (org.apache.kafka.common.utils.AppInfoParser)
[2026-03-30 08:03:56,974] INFO [KafkaServer id=0] started (kafka.server.KafkaServer)
[2026-03-30 08:03:57,086] INFO [zk-broker-0-to-controller-alter-partition-channel-manager]: Recorded new controller, fro
m now on will use node Harini.localdomain:9092 (id: 0 rack: null) (kafka.server.NodeToControllerRequestThread)
[2026-03-30 08:03:57,120] INFO [zk-broker-0-to-controller-forwarding-channel-manager]: Recorded new controller, from now
 on will use node Harini.localdomain:9092 (id: 0 rack: null) (kafka.server.NodeToControllerRequestThread)
[2026-03-30 08:04:33,117] INFO Creating topic raw-sensor-images with configuration {} and initial partition assignment H
ashMap(0 -> ArrayBuffer(0)) (kafka.zk.AdminZkClient)
[2026-03-30 08:04:33,207] INFO [ReplicaFetcherManager on broker 0] Removed fetcher for partitions Set(raw-sensor-images-
0) (kafka.server.ReplicaFetcherManager)
[2026-03-30 08:04:33,256] INFO [LogLoader partition=raw-sensor-images-0, dir=/tmp/kafka-logs] Loading producer state til
l offset 0 with message format version 2 (kafka.log.UnifiedLog$)
[2026-03-30 08:04:33,268] INFO Created log for partition raw-sensor-images-0 in /tmp/kafka-logs/raw-sensor-images-0 with
 properties {} (kafka.log.LogManager)
[2026-03-30 08:04:33,271] INFO [Partition raw-sensor-images-0 broker=0] No checkpointed highwatermark is found for parti
tion raw-sensor-images-0 (kafka.cluster.Partition)
[2026-03-30 08:04:33,273] INFO [Partition raw-sensor-images-0 broker=0] Log loaded for partition raw-sensor-images-0 wit
h initial high watermark 0 (kafka.cluster.Partition)
```

Zookeper:

```
apache.zookeeper.server.ServerCnxnFactory)
[2026-03-30 08:03:18,792] WARN maxCnxns is not configured, using default value 0. (org.apache.zookeeper.server.ServerCnx
nFactory)
[2026-03-30 08:03:18,794] INFO Configuring NIO connection handler with 10s sessionless connection timeout, 2 selector th
read(s), 24 worker threads, and 64 kB direct buffers. (org.apache.zookeeper.server.NIOServerCnxnFactory)
[2026-03-30 08:03:18,805] INFO binding to port 0.0.0.0/0.0.0.0:2181 (org.apache.zookeeper.server.NIOServerCnxnFactory)
[2026-03-30 08:03:18,822] INFO Using org.apache.zookeeper.server.watch.WatchManager as watch manager (org.apache.zookeep
er.server.watch.WatchManagerFactory)
[2026-03-30 08:03:18,822] INFO Using org.apache.zookeeper.server.watch.WatchManager as watch manager (org.apache.zookeep
er.server.watch.WatchManagerFactory)
[2026-03-30 08:03:18,822] INFO zookeeper.snapshotSizeFactor = 0.33 (org.apache.zookeeper.server.ZKDatabase)
[2026-03-30 08:03:18,822] INFO zookeeper.commitLogCount=500 (org.apache.zookeeper.server.ZKDatabase)
[2026-03-30 08:03:18,830] INFO zookeeper.snapshot.compression.method = CHECKED (org.apache.zookeeper.server.persistence.
SnapStream)
[2026-03-30 08:03:18,830] INFO Snapshotting: 0x0 to /tmp/zookeeper/version-2/snapshot.0 (org.apache.zookeeper.server.per
sistence.FileTxnSnapLog)
[2026-03-30 08:03:18,837] INFO Snapshot loaded in 14 ms, highest zxid is 0x0, digest is 1371985504 (org.apache.zookeeper
.server.ZKDatabase)
[2026-03-30 08:03:18,840] INFO Snapshotting: 0x0 to /tmp/zookeeper/version-2/snapshot.0 (org.apache.zookeeper.server.per
sistence.FileTxnSnapLog)
[2026-03-30 08:03:18,841] INFO Snapshot taken in 0 ms (org.apache.zookeeper.server.ZooKeeperServer)
[2026-03-30 08:03:18,856] INFO PrepRequestProcessor (sid:0) started, reconfigEnabled=false (org.apache.zookeeper.server.
PrepRequestProcessor)
[2026-03-30 08:03:18,856] INFO zookeeper.request_throttler.shutdownTimeout = 10000 ms (org.apache.zookeeper.server.Reque
stThrottler)
[2026-03-30 08:03:18,877] INFO Using checkIntervalMs=60000 maxPerMinute=10000 maxNeverUsedIntervalMs=0 (org.apache.zooke
eper.server.ContainerManager)
[2026-03-30 08:03:18,878] INFO ZooKeeper audit is disabled. (org.apache.zookeeper.audit.ZKAuditProvider)
[2026-03-30 08:03:55,442] INFO Creating new log file: log.1 (org.apache.zookeeper.server.persistence.FileTxnLog)
```

7.2 Data Pre-processing / ETL using Spark

After ingestion, the streaming data undergoes ETL (Extract, Transform, Load) operations using PySpark.

Extract: The Base64 encoded image data is extracted from Kafka messages.

Transform: The data is decoded into image format and processed. Basic transformation logic is applied to classify images as defect or non-defect. Additional filtering ensures only valid records are processed.

Load: The processed data is structured into a DataFrame with fields such as product_id, status, and confidence score.

Transformation :

decoded = base64.b64decode(row.value)

Data cleaning is performed to remove corrupted or incomplete data. Aggregation operations can also be used to count the number of defective and non-defective items.

Output Screenshots :

Consumer:

```
26/03/30 09:59:21 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 09:59:31 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 09:59:41 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 09:59:51 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:00:01 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:00:11 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:00:21 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:00:31 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:00:41 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:00:51 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:01:01 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:01:11 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:01:21 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:01:31 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:01:41 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:01:51 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:02:01 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:02:11 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:02:21 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:02:31 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:02:41 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:02:51 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:03:01 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:03:11 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:03:21 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:03:31 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:03:41 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:03:51 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
26/03/30 10:04:01 INFO MicroBatchExecution: Streaming query has been idle and waiting for new data more than 10000 ms.
```

producer:

```
Sent: /mnt/d/BDM_MINI/screw/test/thread_side/018.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_side/019.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_side/020.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_side/021.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_side/022.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/000.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/001.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/002.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/003.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/004.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/005.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/006.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/007.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/008.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/009.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/010.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/011.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/012.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/013.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/014.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/015.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/016.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/017.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/018.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/019.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/020.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/021.png
Sent: /mnt/d/BDM_MINI/screw/test/thread_top/022.png
All images sent!
harini@Harini:/mnt/d/BDM_MINI/producer$
```

7.3 NoSQL Database Integration

MongoDB is used as the NoSQL database to store processed results. It is chosen due to its flexible schema, scalability, and efficient handling of JSON-like data.

The Spark application connects to MongoDB using the PyMongo library. Each processed record is inserted into a collection named "defect_log" in the "quality_control" database.

Example insertion:

```
collection.insert_one({
"product_id": product_id,
"status": status,
"confidence": confidence
})
```

Output Screenshots :

```
test> use quality_control
switched to db quality_control
quality_control> db.defect_log.find().pretty()
[
  {
    _id: ObjectId('69c7d6d9955c9cdc1f1e551d'),
    product_id: '1_0',
    status: 'ok',
    confidence: 0.95
  },
  {
    _id: ObjectId('69c7d6d95e188c0d8b4d3976'),
    product_id: '1_0',
    status: 'ok',
    confidence: 0.95
  },
  {
    _id: ObjectId('69c7d6d9955c9cdc1f1e551e'),
    product_id: '2_0',
    status: 'defect',
    confidence: 0.85
  },
  {
    _id: ObjectId('69c7d6d9955c9cdc1f1e551f'),
    product_id: '2_1',
    status: 'ok',
    confidence: 0.95
  },
  {
    _id: ObjectId('69c7d6d95e188c0d8b4d3977'),
```

```
    product_id: '2_0',
    status: 'defect',
    confidence: 0.85
  },
  {
    _id: ObjectId('69c7d6d9955c9cdc1f1e5520'),
    product_id: '2_2',
    status: 'ok',
    confidence: 0.95
  },
  {
    _id: ObjectId('69c7d6d95e188c0d8b4d3978'),
    product_id: '2_1',
    status: 'ok',
    confidence: 0.95
  },
  {
    _id: ObjectId('69c7d6d9955c9cdc1f1e5521'),
    product_id: '2_3',
    status: 'ok',
    confidence: 0.95
  },
  {
    _id: ObjectId('69c7d6d95e188c0d8b4d3979'),
    product_id: '2_2',
    status: 'ok',
    confidence: 0.95
  },
  {
    _id: ObjectId('69c7d6d95e188c0d8b4d397a'),
```

7.5 Storing Results Back to NoSQL Database

The analytical results and processed data are stored in MongoDB for future reference and reporting. Each record includes product details and detection results.

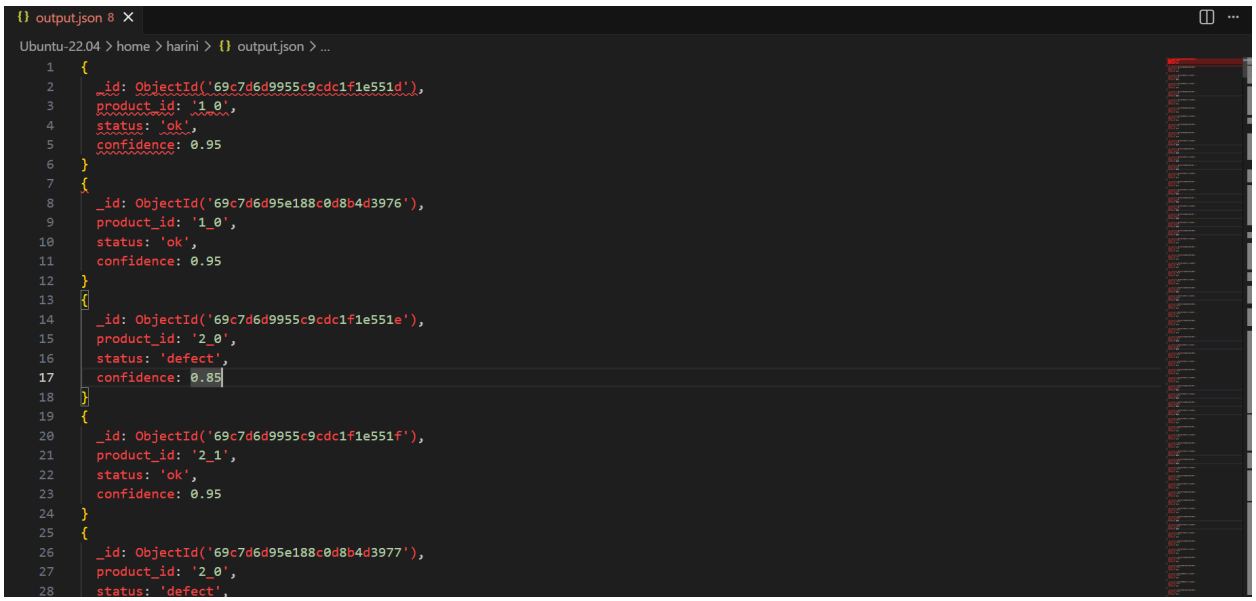
Example stored record:

{
"product_id": "8_2",
"status": "defect",
"confidence": 0.88
}

This ensures persistent storage of processed data and enables further analysis.

Output Screenshots :

Output.json:

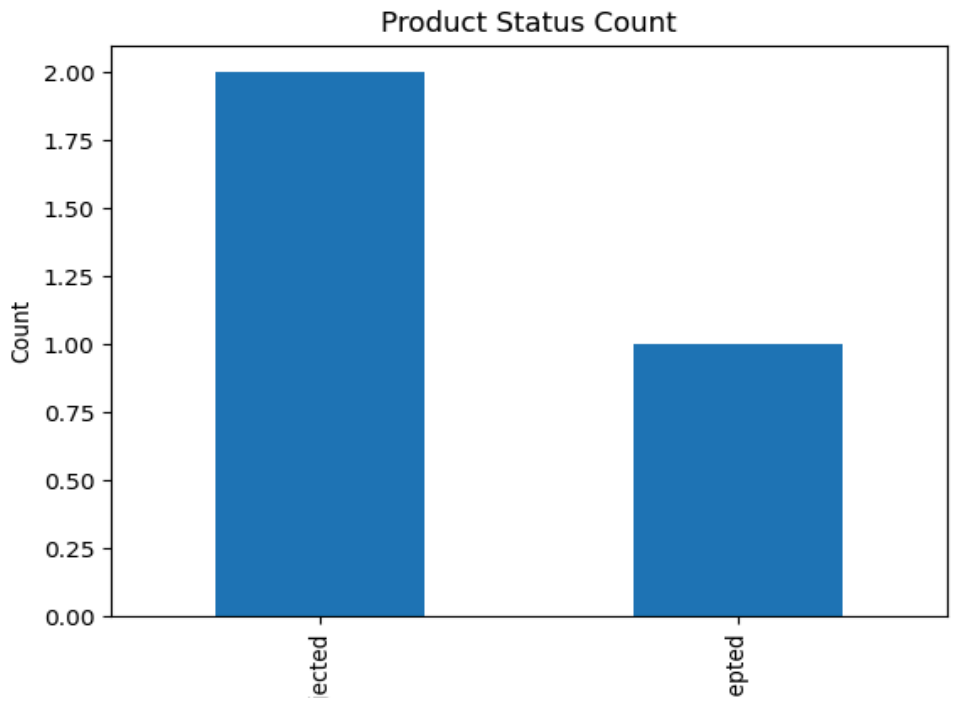

output.json 8
Ubuntu-22.04 > home > harini > output.json > ...
{
_id: ObjectId('69c7d6d9955c9cdc1f1e551d'),
product_id: '1_0',
status: 'ok',
confidence: 0.95
}
{
_id: ObjectId('69c7d6d95e188c0d8b4d3976'),
product_id: '1_0',
status: 'ok',
confidence: 0.95
}
{
_id: ObjectId('69c7d6d9955c9cdc1f1e551e'),
product_id: '2_0',
status: 'defect',
confidence: 0.85
}
{
_id: ObjectId('69c7d6d9955c9cdc1f1e551f'),
product_id: '2_1',
status: 'ok',
confidence: 0.95
}
{
_id: ObjectId('69c7d6d95e188c0d8b4d3977'),
product_id: '2_0',
status: 'defect',

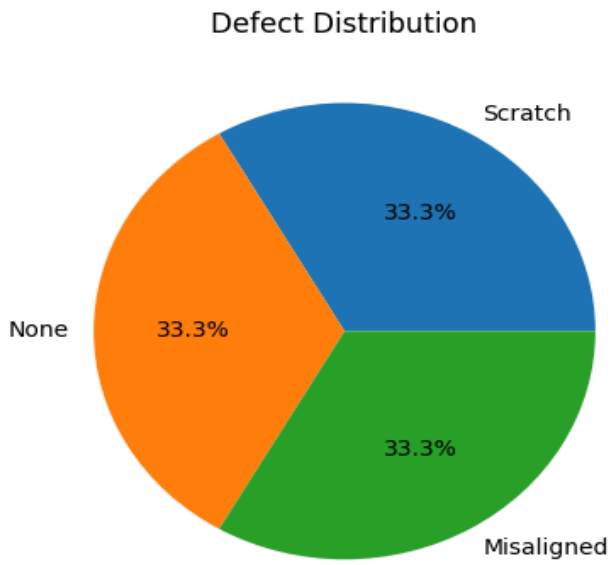
7.6 Data Visualization

Data visualization is performed using Matplotlib to present analytical insights in graphical form.

Bar charts are used to compare defect vs non-defect counts. Line graphs can be used to visualize defect trends over time. These visualizations help in understanding production quality and identifying patterns.

Output Screenshots :



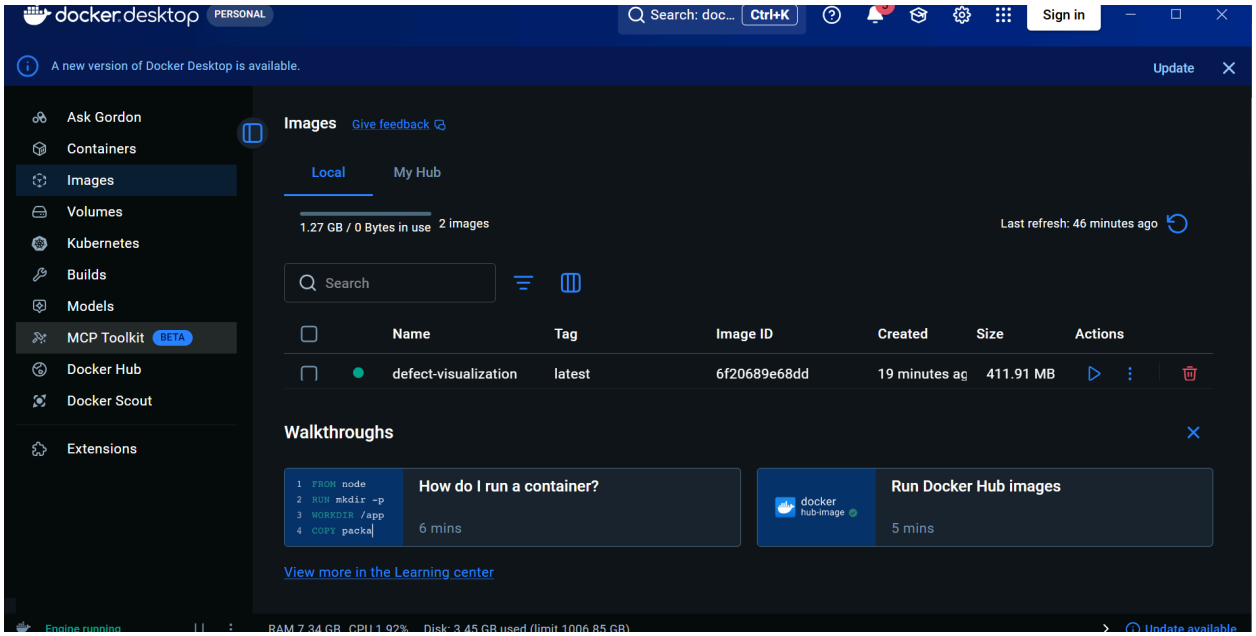


7.7 Deployment of the Big Data Application

The system can be deployed using Docker containers to ensure portability and scalability. Kafka, Spark, and MongoDB services can be containerized and run in isolated environments.

Docker allows easy setup and management of dependencies. Alternatively, the system can be deployed on cloud platforms for large-scale applications.

Output Screenshots :



8. Results and Inference

8.1 Analytical Results Obtained

The system successfully processes streaming image data and classifies products as defect or non-defect in real time. The processed data stored in MongoDB provides a continuous record of product quality. Aggregation results show the count of defective and non-defective items, enabling monitoring of production performance.

8.2 Key Insights Discovered

The analysis helps identify patterns in defect occurrence across different categories of screws. It enables detection of frequent defect types and helps in understanding quality variations in the production process. The system allows continuous monitoring, making it easier to detect issues early and improve manufacturing efficiency.

8.3 Performance Observations

The system demonstrates efficient handling of streaming data with low latency. Spark Structured Streaming processes incoming data continuously without delay, and MongoDB provides fast storage and retrieval of results. The pipeline is scalable and can handle increasing data volume effectively, making it suitable for real-time industrial applications.

9. Impact of the Project

9.1 Human Impact

The project reduces manual inspection efforts and minimizes human errors in quality control. It improves decision-making by providing real-time insights into product quality and enhances safety by ensuring defective products are identified early.

9.2 Societal Impact

The system contributes to smart manufacturing by improving production efficiency and product reliability. It supports automation in industries and can be extended to other domains such as public monitoring, fraud detection, and intelligent systems.

9.3 Ethical Considerations

The system ensures responsible use of data by handling only necessary information. Data privacy must be maintained when dealing with real-world industrial data. Fairness and bias should be considered while designing machine learning models to ensure accurate and unbiased defect detection.

9.4 Sustainable Development

The project supports efficient resource management by reducing waste caused by defective products. Early detection of defects helps minimize material loss and energy consumption, contributing to environmentally sustainable manufacturing practices

10. Conclusion

10.1 Achievements of the Project

The project successfully developed a real-time automated quality control system for detecting defects in manufactured products. A complete big data pipeline was implemented, including data ingestion using Kafka, real-time processing using Spark, and storage using MongoDB. The

system effectively simulated a real-world manufacturing environment and demonstrated continuous processing of streaming data.

10.2 Effectiveness of Big Data Tools Used

The use of Apache Spark enabled efficient and scalable real-time data processing. Kafka provided reliable and high-throughput data streaming, while MongoDB ensured flexible and fast storage of processed results. These tools together created a robust and scalable system capable of handling large volumes of streaming data.

10.3 Insights Obtained

The system provided valuable insights into defect detection and product quality trends. It enabled monitoring of defect rates, identification of defect patterns, and real-time analysis of production data. The project demonstrated how big data technologies can be effectively applied to solve real-world industrial problems.

11. Future Work

- Use of Larger Datasets

  The system can be extended to handle larger and more diverse datasets to improve scalability and robustness.
- Integration with Advanced Machine Learning Models

  Future improvements can include integrating advanced machine learning or deep learning models for more accurate defect detection and classification.
- Development of Real-Time Dashboards

  Interactive dashboards can be developed to visualize real-time analytics and provide better insights to users.
- Advanced Predictive Analytics

  Predictive models can be implemented to forecast defects and enable proactive maintenance in manufacturing systems.
- Cloud-Based Deployment

  The system can be deployed on cloud platforms to enhance scalability, availability, and real-world applicability.

12. Learning Outcomes

- Build a Big Data Processing Pipeline
  Gained knowledge in designing and implementing a complete big data pipeline from data ingestion to analysis.
- Implement Streaming Analytics Using Spark
  Learned how to use Spark Structured Streaming for processing real-time data efficiently.
- Integrate Spark with NoSQL Databases
  Understood how to connect Spark with MongoDB for storing and retrieving processed data.
- Deploy Big Data Applications
  Acquired experience in deploying applications using containerization or cloud-based environments.
- Visualize Analytical Results
  Developed skills in visualizing data using tools such as Matplotlib for better interpretation of results.

13. References

1. Apache Spark Documentation. Available at: https://spark.apache.org/docs/latest/
2. Apache Kafka Documentation. Available at: https://kafka.apache.org/documentation/
3. MongoDB Documentation. Available at: https://www.mongodb.com/docs/
4. Kaggle Dataset Repository – Screw Defect Dataset. Available at: https://www.kaggle.com/
5. Dean, J. and Ghemawat, S. (2008). MapReduce: Simplified Data Processing on Large Clusters. Communications of the ACM.
6. Zaharia, M. et al. (2016). Apache Spark: A Unified Engine for Big Data Processing. Communications of the ACM.
7. White, T. (2015). Hadoop: The Definitive Guide. O'Reilly Media.
8. Karau, H., Konwinski, A., Wendell, P., and Zaharia, M. (2015). Learning Spark. O'Reilly Media.

9. Han, J., Kamber, M., and Pei, J. (2011). Data Mining: Concepts and Techniques. Morgan Kaufmann.
10. OpenCV Documentation. Available at: https://docs.opencv.org/
11. Matplotlib Documentation. Available at: https://matplotlib.org/stable/contents.html

GitHub Link:

Click here to view the project

https://github.com/harriinnii/BigData_product_defect_detection.git